



FACULTY OF COMPUTERS AND ARTIFICIAL INTELLIGENCE, CAIRO UNIVERSITY

IT443: Image Processing

Project: Face Recognition

Course Instructor:

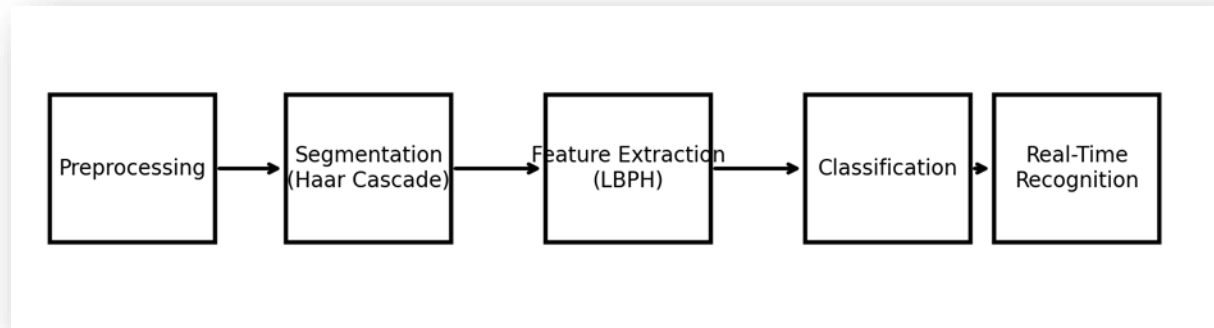
Dr. Asmaa Ahmed

Prepared by :

Name	ID	Email
Hady Hassan Ahmed	20230454	20230454@stud.fci-cu.edu.eg
Abdallah Kareem Abdallah	20230228	20230228@stud.fci-cu.edu.eg
NourEldin Mohamed Amin	20230444	20230444@stud.fci-cu.edu.eg

Project Goal:

This project aims to design and implement a complete **Real-Time Face Recognition System** using Digital Image Processing techniques. The system satisfies the core course requirements by following a standard image processing pipeline that includes:



The application captures video frames from a webcam, detects and isolates human faces, enhances the facial images, extracts distinctive features, and identifies individuals in real time.

Pipeline:

1. Image Acquisition

Image Acquisition is the first stage of the proposed Face Recognition System. In this step, facial images are collected using a webcam connected to the computer. A custom Python script (`data_collector.py`) captures multiple images for each team member and dataset for different people under different conditions, including variations in facial expressions, head orientations, and lighting environments.

2. Preprocessing

After acquiring the facial images, a preprocessing stage is applied to enhance image quality and prepare the dataset for training. First, each image is converted to grayscale and the face region is detected using the Haar Cascade classifier. The largest detected face is cropped and resized to a fixed size of **200 × 200 pixels** to ensure consistency across all samples.

To improve image contrast and reduce the effect of uneven lighting, **Contrast Limited Adaptive Histogram Equalization (CLAHE)** is applied with a clip limit of 2.0 and a tile grid size of 8 × 8. This technique enhances important facial details while preventing excessive amplification of noise.

To increase the size and diversity of the dataset, several data augmentation techniques are performed, including horizontal flipping, small rotations (−10°, −5°, +5°, and +10°), and brightness

adjustments. These transformations generate additional training samples that help the recognition model become more robust to changes in pose and illumination.

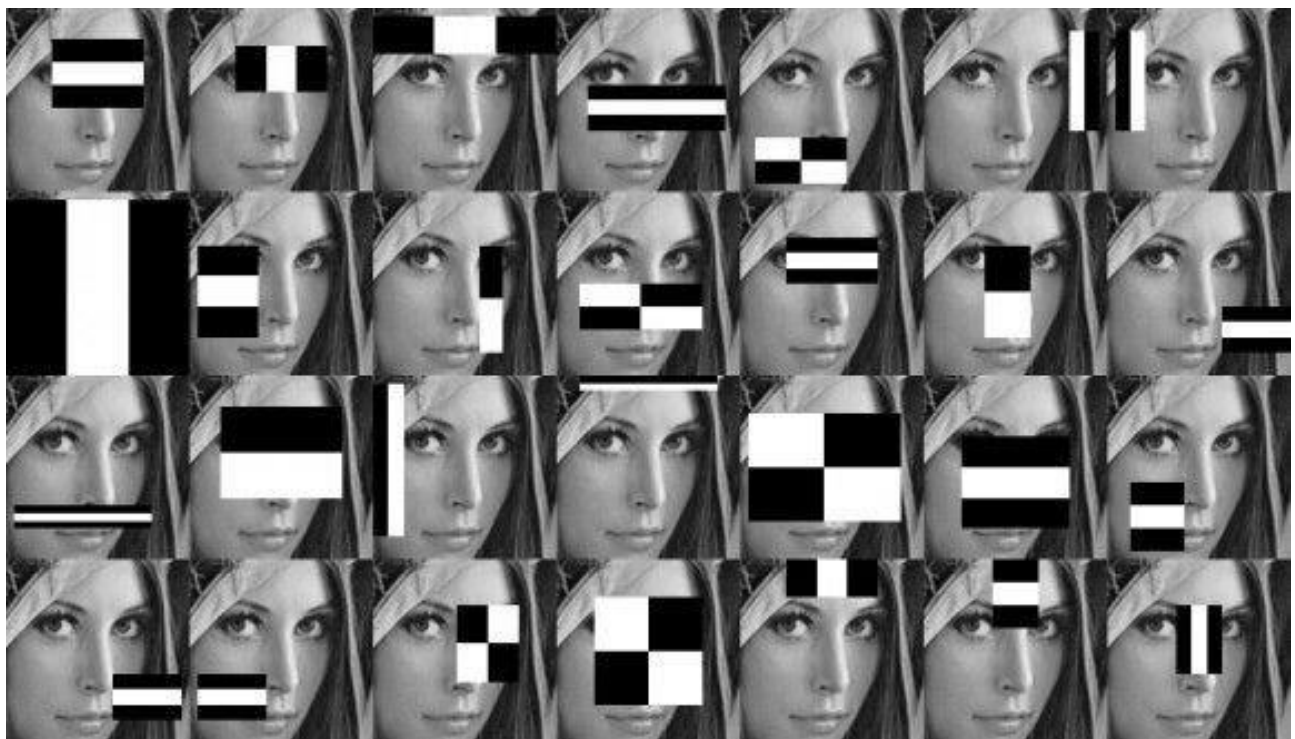
The resulting processed and augmented images are stored in the `processed_dataset/` directory and used as the final input for the feature extraction and training stages

3. Segmentation

The segmentation stage is implemented through face detection and region isolation using the **Haar Cascade classifier**. Although classical image segmentation typically refers to partitioning an image into meaningful regions, in face recognition the primary objective is to separate the facial area from the surrounding background.

The Haar Cascade algorithm scans each webcam frame and identifies rectangular regions that are likely to contain a human face. Once a face is detected, the corresponding bounding box is used to crop the facial region and discard irrelevant background information such as walls, clothing, and other objects. If multiple faces are detected, the largest face is selected as the region of interest.

This cropped face represents the segmented output of the system and is passed to the preprocessing and feature extraction stages. By isolating only the relevant facial region, the system reduces computational complexity and ensures that subsequent processing focuses exclusively on discriminative facial features.



4. Feature Extraction

In the initial implementation, the system used **the Local Binary Patterns Histograms (LBPH)** algorithm for feature extraction. **LBPH** represents each face by analyzing local texture patterns and generating histograms that describe the distribution of these patterns across the image. This approach is computationally efficient and performs well on small datasets; however, several limitations were observed during experimentation.

The main drawback of **LBPH** was its sensitivity to significant variations in lighting conditions, head pose, and facial expressions. In addition, its handcrafted texture descriptors were not sufficiently discriminative when dealing with similar-looking individuals, which resulted in reduced recognition accuracy and occasional misclassifications during real-time testing.

To overcome these limitations, the feature extraction stage was upgraded to use [**FaceNet**](#), **a deep learning-based facial embedding model**. **FaceNet** converts each detected face into a compact 128-dimensional feature vector (embedding) that captures high-level and highly discriminative facial characteristics. These embeddings place images of the same person close together in feature space, while images of different individuals are separated by larger distances.

Compared with **LBPH**, **FaceNet** provides substantially better robustness to changes in illumination, pose, facial expression, and image quality. It also generalizes more effectively to unseen samples and achieves significantly higher recognition accuracy, especially when distinguishing between individuals with similar facial features.

Aspect	LBPH (Initial Approach)	FaceNet (Final Approach)
Method Type	Classical texture-based feature extraction	Deep learning-based facial embeddings
Feature Representation	Local Binary Pattern histograms	128-dimensional embedding vector
Recognition Accuracy	Moderate	High
Robustness to Lighting and Pose	Limited	Excellent
Similar Face Discrimination	May confuse similar-looking individuals	Highly discriminative
Computational Cost	Low	Moderate
Main Advantages	Simple, fast, and easy to implement	More accurate and robust
Main Limitations	Sensitive to lighting, pose, and expressions	Requires more dependencies and computation
Experimental Result	Produced inconsistent recognition in challenging conditions	Achieved stable and accurate real-time recognition
Final Decision	Used in the initial prototype	Selected as the final feature extraction method

5. Classification

In the classification stage, the feature vectors extracted by [FaceNet](#) are used to identify individuals by comparing them with previously stored embeddings. During enrollment, each processed face is converted into a 128-dimensional embedding and associated with a corresponding label (person name or ID).

During recognition, the embedding of the detected face is computed and compared with all registered embeddings using a distance metric such as Euclidean distance. The identity corresponding to the smallest distance is selected as the predicted class.

A similarity threshold is then applied to determine whether the detected face belongs to a known person. If the minimum distance is below the predefined threshold, the corresponding identity is assigned; otherwise, the face is labeled as **“Unknown”**.

This approach provides significantly higher accuracy and robustness than traditional methods such as LBPH, particularly under variations in lighting, pose, and facial expressions. By leveraging deep facial embeddings, the system achieves reliable and efficient real-time face classification

5. Real-Time Recognition

The system operates in real time using a webcam feed, where each captured frame is processed continuously. Faces are first detected using the Haar Cascade classifier and then passed through the same preprocessing pipeline to ensure consistency with the training data. Next, [FaceNet](#) extracts a 128-dimensional embedding from each detected face, and the classification module compares this embedding with the stored representations of registered individuals.

If the minimum distance is below a predefined threshold, the corresponding person's name is displayed on the screen; otherwise, the face is labeled as **Unknown**. This process is repeated for every frame, enabling accurate and efficient real-time face recognition

Challenges and Limitations

During the development of the system, several challenges were encountered:

- **Lighting variations:** Strong or weak lighting affected detection and recognition accuracy.
- **Pose changes:** Large head rotations or side profiles reduced detection performance.
- **Similar faces:** People with similar facial features sometimes produced close confidence scores.
- **Small dataset size:** Limited training images reduced model generalization.
- **Haar Cascade limitations:** Detection may fail with occlusions or non-frontal faces.
- **Threshold tuning:** Choosing the correct confidence threshold for “Unknown” classification was difficult and affected accuracy.

These challenges were partially mitigated using preprocessing, augmentation, and parameter tuning, but they remain inherent limitations of classical face recognition systems.

Team Project Summary:

Person	Role	Deliverables
Hady Hassan	Data & Preprocessing	• <code>data_collector.py</code>, for capture images for each member in the team • <code>image_enhancer.py</code>, for removing noise and more advanced enhancement and data augmented
NourEldin Mohamed	Modelling & Training	• <code>trainer.py</code>, using LBPH algorithm for extract feature and training
Abdallah Kareem	System Integrator	• <code>face_recognizer.py</code>, integrate all parts with one file • Report Summary